

```
//2. 전처리기
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

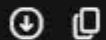
이번 코드는 흐름을 아주 예리하게 잘 잡아내셨어요! 입력을 받고 아래로 내려가기 전에 `if (choice == 6)` 으로 `break` 를 걸어서 `while` 무한 루프를 탈출하도록 흐름을 정확하게 설계하셨습니다.

하지만 딱 한 가지, 프로그램이 영원히 끝나지 않거나 수치가 꼬이게 되는 심각한 논리적 모순(버그)이 하나 남아있습니다.

🔥 현재 코드의 결정적인 문제점: "첫 루프의 함정"

코드를 위에서부터 순서대로 읽는 컴퓨터의 시선으로 따라가 볼게요.

C



```
// 1. 처음 프로그램이 켜지면 choice는 0으로 시작합니다.
int choice = 0;

while(1)
{
    // 2. [첫 번째 루프 시작] choice가 0이므로 이 if문은 그냥 통과합니다.
    if (choice == 6) { break; }

    // 3. 사용자에게 메뉴를 보여주고 입력을 받습니다.
    // 여기서 사용자가 종료하고 싶어서 '6'을 입력했다고 가정해 볼게요!
    printf("1. 별가루 폭식 ... 6. 나가기\n선택: ");
    scanf("%d", &choice); // 변수 choice에 '6'이 저장됨

    switch(choice)
    {
        // ... 1~5번 케이스들 ...
        default:
            // 4. choice가 6이므로 1~5 어디에도 해당하지 않아 default로 들어옵니다!
            printf("\n[ 잘못된 선택입니다. 1~6 사이의 숫자를 눌러주세요. ]\n");
            break;
    }

    // 5. switch문을 빠져나와서 하필 스텟 업데이트 함수를 실행합니다.
    update_and_adjust_stats(current_mass, stardust_count);

    // 6. [두 번째 루프 시작] 다시 위로 올라옵니다.
    // 7. 이제 choice가 6이므로 드디어 여기서 break를 만나 while 루프를 탈출합니다!
}

}
```

이로 인해 발생하는 2가지 버그:

1. **황당한 출력 오류:** 사용자가 종료하려고 정상적으로 6 을 눌렀는데, 화면에는 느닷없이 [잘못된 선택입니다. 1~6 사이의 숫자를 눌러주세요.] 라는 경고문이 뜹니다. 사용자는 "6번 나가기 누르라며?" 하고 당황하게 되겠죠.
2. **종료 스텟 오염:** 루프를 즉시 나가지 못하고 아래로 내려가 `update_and_adjust_stats` 함수를 쓸데없이 한 번 더 실행하면서 최종 스텟 계산이 한 번 더 꼬이게 됩니다.

아 그리고 반복문으로 바꾸게 되면 데이터 보정을 매번 선택할 때마다 해주는 게 낫겠지

맞습니다! 정확한 지적이세요. 🙌

행동을 선택할 때마다 스탯(에너지, 스트레스, 애정 등)이 계속 실시간으로 변하기 때문에, 데이터 보정(if 문 제한)과 수치 계산은 매번 선택이 끝나는 순간(반복문이 한 바퀴 돌 때마다) 바로 해주는 것이 훨씬 좋습니다.

만약 이걸 매번 안 해주면 다음과 같은 심각한 버그들이 발생하게 됩니다.

🔥 매번 보정을 안 하면 생기는 문제들 (예시)

1 스트레스 음수 버그

- 초기 스트레스가 0 인 상태에서 2번 쓰다듬기(`stress -= 15`)를 연속으로 3번 하면 스트레스가 -45% 가 되어버립니다.

2 에너지 오버플로우

- 에너지가 최대치인 200% 를 뚫고 300% , 400% 까지 무한정 늘어날 수 있습니다.

3 스탯 리포트 싱크 불일치

- 1번(폭식)이나 5번(탐사)을 해서 질량이나 별가루가 바뀌었는데, 계산을 매번 안 해주면 토끼 리포트에는 계속 옛날 질량과 밀도가 출력됩니다.